

TP n°5 — Transactions et contrôle de concurrence

Exercice 1 : Atomicité d'une transaction

Question 1 — Création de la table transaction (Session S1)

Dans la session S1, après avoir désactivé l'autocommit, la table transaction a été créée avec le script suivant :

```
SET AUTOCOMMIT OFF
CREATE TABLE transaction (
    idTransaction VARCHAR2(44),
    valTransaction NUMBER(10)
);
```

La table a été créée avec succès (message : « Table créée »). La session S2 a été ouverte en parallèle avec le même réglage SET AUTOCOMMIT OFF.

Question 2 — Insertions, modification, suppression et ROLLBACK (Session S2)

Dans la session S2, les opérations suivantes ont été exécutées :

```
INSERT INTO transaction VALUES ('T1', 100);
INSERT INTO transaction VALUES ('T2', 200);
INSERT INTO transaction VALUES ('T3', 300);
UPDATE transaction SET valTransaction = 500 WHERE idTransaction = 'T1';
DELETE FROM transaction WHERE idTransaction = 'T2';
```

Après ces opérations, un SELECT affichait les lignes T1 (valeur 500) et T3 (valeur 300). Ensuite, un ROLLBACK a été exécuté :

```
ROLLBACK;
```

Résultat observé : le SELECT suivant n'a retourné aucune ligne (« aucune ligne sélectionnée »). Toutes les modifications ont été annulées, confirmant le principe d'atomicité : soit toutes les opérations sont validées, soit aucune ne l'est.

Question 3 — Insertions et fermeture avec quit (Session S2)

De nouvelles lignes ont été insérées dans la session S2 :

```
INSERT INTO transaction VALUES ('T10', 1000);
INSERT INTO transaction VALUES ('T11', 1100);
quit
```

Observation depuis la session S1 : un SELECT * FROM transaction affiche bien T10 (1000) et T11 (1100).

Explication : la commande quit (ou exit) dans SQL*Plus provoque un COMMIT implicite. Les données ont donc été automatiquement validées à la fermeture de la session, et sont visibles par les autres sessions.

Question 4 — Fermeture brutale de sqlplus (Session S1)

Dans la session S1, deux nouvelles lignes ont été insérées :

```
INSERT INTO transaction VALUES ('T43', 4000);  
INSERT INTO transaction VALUES ('T65', 9000);
```

Puis sqlplus a été fermé brutalement (via la croix de la fenêtre Windows).

Après reconnexion : un `SELECT * FROM transaction` ne montre que T10 et T11. Les lignes T43 et T65 n'ont pas été conservées.

Explication : la fermeture brutale ne déclenche pas de `COMMIT` implicite. Oracle exécute un `ROLLBACK` automatique pour toute transaction non validée interrompue de manière anormale. Les données non validées sont donc perdues.

Question 5 — Modification de structure (`ALTER TABLE`) et `ROLLBACK`

Dans une nouvelle session, des lignes ont été insérées, puis la structure de la table a été modifiée :

```
INSERT INTO transaction VALUES ('T8', 8000);  
INSERT INTO transaction VALUES ('T4', 5600);  
ALTER TABLE transaction ADD val2transactin NUMBER(10);
```

Résultat : la table a été modifiée (colonne `val2transactin` ajoutée). Un `SELECT` affiche toutes les lignes : T10, T11, T8, T4 avec la nouvelle colonne.

Après un `ROLLBACK`, un nouveau `SELECT` affiche toujours les mêmes 4 lignes avec la colonne `val2transactin`.

Explication : en Oracle, les commandes DDL (comme `ALTER TABLE`) provoquent un `COMMIT` implicite avant et après leur exécution. Il est donc impossible d'annuler un `ALTER TABLE` par un `ROLLBACK`. Le `ROLLBACK` n'a eu aucun effet sur la structure ni sur les données validées implicitement par le DDL.

La commande `DESC transaction` confirme la structure finale :

```
IDTRANSACTION  VARCHAR2(44)  
VALTRANSACTION NUMBER(10)  
VAL2TRANSACTIN NUMBER(10)
```

Question 6 — Conclusions et définitions

Session

Une session correspond à une connexion active d'un utilisateur à Oracle. Elle débute lors de la connexion (`sqlplus user/password`) et se termine à la déconnexion (`quit`, `exit`, ou fermeture). Plusieurs sessions peuvent être ouvertes simultanément par le même utilisateur ou des utilisateurs différents. Chaque session dispose de son propre espace de travail transactionnel.

Transaction

Une transaction est une séquence d'opérations SQL (`INSERT`, `UPDATE`, `DELETE`) traitée comme une unité logique indivisible. Elle commence implicitement après la dernière validation ou annulation, et se termine par un `COMMIT` (validation) ou un `ROLLBACK` (annulation). Une transaction respecte les propriétés ACID :

- Atomicité : tout ou rien
- Cohérence : la base reste dans un état cohérent
- Isolation : les modifications non validées sont invisibles aux autres sessions
- Durabilité : après un `COMMIT`, les données sont permanentes

Valider ou annuler une transaction

- COMMIT : valide définitivement toutes les modifications de la transaction en cours. Elles deviennent visibles par les autres sessions et ne peuvent plus être annulées.
- ROLLBACK : annule toutes les modifications effectuées depuis le dernier COMMIT. La base revient à son état précédent.
- COMMIT implicite : déclenché automatiquement par Oracle lors d'un DDL (ALTER, CREATE, DROP) ou d'une déconnexion normale (quit/exit).
- ROLLBACK implicite : déclenché par Oracle en cas de fermeture brutale ou d'erreur grave.

Exercice 2 : Transactions concurrentes

L'objectif de cet exercice est d'étudier les problèmes liés à l'exécution concurrente de transactions et l'impact des niveaux d'isolation.

Création du schéma

Deux tables ont été créées pour simuler une gestion de réservations de billets d'avion :

```
CREATE TABLE vol (idVol VARCHAR2(44), capaciteVol NUMBER(10),  
nbrPlacesReserveesVol NUMBER(10));  
CREATE TABLE client (idClient VARCHAR2(44), prenomClient VARCHAR2(11),  
nbrPlacesReserveesClient NUMBER(10));
```

Les données initiales insérées :

- Vol V1 : capacité 100, 0 places réservées
- Client C1 (Ali) : 0 places réservées
- Client C2 (Sara) : 0 places réservées

Partie 1 — Isolation des transactions (READ COMMITTED)

Observation : visibilité des mises à jour

Dans la session T1 (niveau d'isolation par défaut READ COMMITTED), une réservation a été effectuée pour C1 (1 place) sans COMMIT :

```
UPDATE vol SET nbrPlacesReserveesVol = nbrPlacesReserveesVol + 1 WHERE  
idvol='vol1';  
UPDATE client SET nbrPlacesReserveesClient = nbrPlacesReserveesClient +  
1 WHERE idclient = 'C1';
```

Observations :

- T1 voit immédiatement ses propres modifications (vol : 1 place, C1 : 1 place).
- T2, interrogeant les mêmes tables, voit toujours les valeurs initiales (vol : 0, C1 : 0, C2 : 0). Les modifications non validées de T1 sont invisibles pour T2.

Cela illustre la propriété d'isolation des transactions : les modifications d'une transaction ne sont visibles par les autres sessions qu'après un COMMIT.

ROLLBACK de T1

Un ROLLBACK a été exécuté dans T1. Résultat : la base revient à l'état initial (vol : 0, C1 : 0). Pour T2, T1 n'a jamais existé. Un COMMIT ou ROLLBACK supplémentaire dans T1 n'a aucun effet car une nouvelle transaction a déjà démarré.

COMMIT de T1 et durabilité

La transaction T1 a été relancée, puis validée par COMMIT. Observations :

- Après le COMMIT, un ROLLBACK ne peut pas annuler les données (vol : 1). C'est la propriété de durabilité.
- T2 voit désormais les modifications de T1 (vol : 1, C1 : 1). Cela confirme que le niveau d'isolation par défaut d'Oracle est READ COMMITTED : une transaction voit les données validées par les autres transactions au moment où elle les lit.
- En mode REPEATABLE READ, T2 n'aurait pas vu ces mises à jour car elle a démarré avant le COMMIT de T1.

Partie 2 — Le problème des mises à jour perdues (READ COMMITTED)

L'exécution entrelacée suivante a été reproduite (réinitialisation des données) :

T1 (Ali — 2 places)	T2 (Sara — 3 places)
SELECT vol → 0, client C1 → 0	SELECT vol → 0, client C2 → 0
UPDATE vol SET places = 2 UPDATE client C1 = 2 COMMIT	UPDATE vol SET places = 3 UPDATE client C2 = 3 COMMIT

Résultat final observé : C1 (Ali) = 2 places, C2 (Sara) = 3 places, vol V1 = 3 places réservées.

Incohérence constatée : le vol affiche seulement 3 places réservées alors qu'en réalité 2 + 3 = 5 billets ont été vendus. La mise à jour de T1 sur le vol a été écrasée par T2, car T2 avait lu la valeur 0 avant le COMMIT de T1 et a écrit 3 par-dessus la valeur 2 validée par T1.

C'est le problème classique des mises à jour perdues (Lost Update), causé par un niveau d'isolation insuffisant (READ COMMITTED) dans un contexte d'accès concurrent.

Partie 3 — Isolation complète : mode SERIALIZABLE

Les deux sessions ont été réinitialisées et le niveau d'isolation SERIALIZABLE a été activé :

```
SET TRANSACTION ISOLATION LEVEL SERIALIZABLE;
```

Le même scénario concurrent a été reproduit :

- T1 lit vol V1 (0 place) et client C1 (0 place).
- T2 lit vol V1 (0 place) et client C2 (0 place).
- T1 met à jour : vol → 2, C1 → 2. COMMIT effectué.
- T2 tente de mettre à jour le vol → Oracle détecte un conflit de sérialisation et retourne l'erreur suivante :

```
ORA-08177: impossible de sérialiser l'accès pour cette transaction
```

Résultat : T2 est rejetée. La base reste dans un état cohérent. Seule T1 a été validée (vol : 2, C1 : 2).

L'algorithme SERIALIZABLE d'Oracle garantit qu'aucune transaction ne peut modifier une donnée qui a été modifiée et validée par une autre transaction après le début de la première lecture. Cela élimine les mises à jour perdues mais peut entraîner des rejets de transactions en cas de conflit.

Question finale — Algorithme Oracle vs verrouillage à deux phases (2PL)

L'algorithme implémenté dans Oracle en mode SERIALIZABLE n'est pas le verrouillage à deux phases (2PL) classique vu en cours.

Oracle utilise une technique appelée MVCC (Multi-Version Concurrency Control) avec détection de conflits au moment du COMMIT. Chaque transaction travaille sur un snapshot

cohérent de la base pris au début de la transaction. Au moment du COMMIT, Oracle vérifie si des données lues ou modifiées ont été modifiées par une autre transaction validée entre-temps. Si c'est le cas, Oracle rejette la transaction avec l'erreur ORA-08177.

Différences avec le 2PL :

- 2PL (verrouillage à deux phases) : pose des verrous en phase croissante, les libère en phase décroissante. Bloque les autres transactions qui tentent d'accéder aux données verrouillées. Peut provoquer des interblocages (deadlocks).
- Oracle SERIALIZABLE (MVCC) : ne pose pas de verrous en lecture. Travaille sur des snapshots. Rejette les transactions au COMMIT en cas de conflit, plutôt que de les bloquer. Les lectures ne bloquent jamais les écritures et vice versa.